

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-aceaeaa2f114ce683.jsonl`
- SHA-256: `c2293ac7ef2ce6f63d77569f318b0b0d665338d323f7aa0468f32e1f3ca3b6a2`
- Source modified: `2026-06-22T15:07:00+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T15:05:03.851Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: dispatch-correctness

Severity: minor

Title: Cloud segments returned/finalized via query_play_segments with no ORDER BY ? start-order is only implicit (rowid)

File: backend/main.py:522

Detail: On the cloud path the segments list is rebuilt by round-tripping through db_instance.query_play_segments(video_id, {}), which (database.py:564-612) has NO ORDER BY clause, so rows come back in SQLite's default scan order (rowid asc = insertion order = CSV read order = start order, in practice). The in-process path instead returns the segmenter's own ordered result.value directly. The cloud reel ordering therefore depends on the implicit rowid==insertion invariant holding (it does today because _write_intervals_csv emits start-ordered rows and DictReader preserves file order). This is fragile: query_candidate_segments deliberately adds 'ORDER BY start_time' (database.py:524) but query_play_segments does not, so any future query-plan change or a non-start-ordered intervals.csv would silently scramble the cloud reel order.

Fix: Sort the cloud segments explicitly by start_time before building the response/finalizing (e.g. sorted(..., key=lambda s: s['start_time'])), or add an ORDER BY start_time to query_play_segments so both paths are deterministically start-ordered.

2. assistant (2026-06-22T15:05:07.175Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repo.

3. user (2026-06-22T15:05:10.376Z)

```
backend/main.py | 100 ++++++
tests/test_api_cloud_dispatch.py | 212 ++++++
2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:05:13.361Z)

Let me look at the diff and the cited code.

5. user (2026-06-22T15:05:14.556Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
```

```

@@ -1,10 +1,12 @@
import os
import sys
import argparse
+import csv
import json
import logging
import shutil
import socket
+import tempfile
import threading
import uuid
import webbrowser

@@ -436,6 +438,94 @@ def download_reel(filename: str):
    return FileResponse(path, media_type=media, filename=name)

+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
+    bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
+    produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
+    (cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                   "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                             play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
+    ``deployment.compute_target
+    == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
+    intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
+    ``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
+    ``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend

```

```

+     compute = get_compute_backend(global_config)
+     if compute is None:
+         return None # default-OFF ? run the in-process segmenter below (unchanged)
+
+     storage_cfg = getattr(global_config, "storage", None)
+
+     def _failed(msg: str) -> Dict[str, Any]:
+         # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+         return {"video_id": video_id, "status": "failed", "message": msg,
+                 "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+     try:
+         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+     except ValueError as e:
+         return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+     if input_ref.backend == "local":
+         return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+
+                         "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+     if not out_root:
+         return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+
+                         "? that's where the Cloud Run job writes the result.")
+
+     notify("dispatching (cloud_run)")
+     logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+     spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+     try:
+         result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
+     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+         return _failed(f"cloud dispatch error: {e}")
+     if result.returncode != 0:
+         return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+     notify("ingesting (cloud_run)")
+     storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+     _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+     segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+     _finalize_reingest(video_id, segments, play_wm, cand_wm)
+     return {"video_id": video_id, "status": "success",
+             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+             "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
+         """The full hash ? validate ? segment ? report pipeline for one video.
@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
+             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
+             # if the run yields segments; otherwise drop the new partial rows and keep the old.

```

```

        play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+       # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+       # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+       # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+       # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+       # they're dropped on the cloud path in this slice ? a documented follow-up.)
+       remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+       if remote is not None:
+           segmentation_ran = True
+           response = remote
+           return response
        segmenter = segmenter_cls(db_instance, global_config)
        try:
            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)

```

6. assistant (2026-06-22T15:05:17.054Z)

Now let me look at the cited line 522 and the `query\_play\_segments` function in database.py, and compare with the in-process path and `query\_candidate\_segments`.

7. user (2026-06-22T15:05:18.336Z)

```

        return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
                        "can't be read by the Cloud Run job. Put the video in your bucket
first.")
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        if not out_root:
            return _failed("cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
                            "? that's where the Cloud Run job writes the result.")

        notify("dispatching (cloud_run)")
        logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)", req.video_path,
out_root)
        spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
        try:
            result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
        except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure, not
a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}")
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).")

        notify("ingesting (cloud_run)")
        storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
        _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
        segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
        _finalize_reingest(video_id, segments, play_wm, cand_wm)
        return {"video_id": video_id, "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
                "results": [r.model_dump() for r in val_results], "play_segments": segments}

```

```
def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

    This is the former synchronous /api/process body, unchanged in behavior, now run on
    the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
    the sync endpoint used to return (UI compatibility); the per-video observability
    report is still emitted in `finally:` and grafted as response["reports"].

    `progress` is an optional callable(stage: str) used to surface coarse job progress.
    Raises on unexpected internal errors ? the caller records those as a job failure
    (after this function has already restored the pre-run segments, see below).
    """
```

8. assistant (2026-06-22T15:05:21.312Z)

The cited line 522 is indeed the cloud-path rebuild: `segments = [s for s in db\_instance.query\_play\_segments(video\_id, {})] if int(s.get("id", 0)) > play\_wm]`. Now let me check `query\_play\_segments` and `query\_candidate\_segments` in database.py.

9. user (2026-06-22T15:05:22.522Z)

```
        logger.error(f"Failed to add candidate segment: {e}")
        return None

def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
    """Record that a candidate detection was confirmed as a given play_segment."""
    return self._execute_write(
        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
        (segment_id, candidate_id),
        f"Failed to link candidate {candidate_id} to segment {segment_id}",
    )

def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                             only_unlabeled: bool = False) -> List[Dict[str, Any]]:
    """Fetch candidate detections for the annotation / fine-tuning workflow.
    `stats` and `detection_params` are deserialized from JSON."""
    query = "SELECT * FROM candidate_segments WHERE video_id = ?"
    params: List[Any] = [video_id]

    if detector:
        query += " AND detector = ?"
        params.append(detector)
    if only_unlabeled:
        query += " AND human_label IS NULL"

    query += " ORDER BY start_time"

    candidates = []
    with self._connect() as conn:
        rows = conn.cursor().execute(query, params).fetchall()
        for row in rows:
            d = dict(row)
            d["stats"] = json.loads(d["stats"] or "{}")
            d["detection_params"] = json.loads(d["detection_params"] or "{}")
            candidates.append(d)
    return candidates

def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
    with self._connect() as conn:
        row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
        (video_id,)).fetchone()
        if row:
            d = dict(row)
            d["validation_results"] = json.loads(d["validation_results"] or "[]")
            return d
```

```

        return None

def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
    """All indexed videos, most-recently-added first. Orders by rowid (a monotonic insertion
    counter, since id is a TEXT PK) rather than the 1-second-granularity processed_at, which
    would tie within a batch. Mirrors get_video's row shape (validation_results parsed).
Powers
    GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
    sql = "SELECT * FROM videos ORDER BY rowid DESC"
    params: tuple = ()
    if limit is not None:
        sql += " LIMIT ?"
        params = (int(limit),)
    with self._connect() as conn:
        rows = conn.cursor().execute(sql, params).fetchall()
    out: List[Dict[str, Any]] = []
    for row in rows:
        d = dict(row)
        d["validation_results"] = json.loads(d["validation_results"] or "[]")
        out.append(d)
    return out

def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
    """
    Dynamically queries play segments based on filters:
    - duration_min: float
    - server: string
    - receiver: string
    - event_type: string (e.g. smash, foul)
    """
    query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
    params = [video_id]

    if filters.get("duration_min"):
        query += " AND r.duration >= ?"
        params.append(filters["duration_min"])

    if filters.get("server"):
        query += " AND r.server = ?"
        params.append(filters["server"])

    if filters.get("receiver"):
        query += " AND r.receiver = ?"
        params.append(filters["receiver"])

    if filters.get("event_type"):
        # Check if there is an event of a specific type inside this segment
        query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND e.type =
?)"
        params.append(filters["event_type"])

    segments_list = []
    with self._connect() as conn:
        cursor = conn.cursor()
        rows = cursor.execute(query, params).fetchall()

        for row in rows:
            r_dict = dict(row)
            r_id = r_dict["id"]
            r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

            # Fetch associated events
            event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()

```

```

        r_dict["events"] = []
        for er in event_rows:
            e_dict = dict(er)
            e_dict["details"] = json.loads(e_dict["details"] or "{}")
            r_dict["events"].append(e_dict)

        segments_list.append(r_dict)

    return segments_list

def clear_video_data(self, video_id: str):
    with self._connect() as conn:
        conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
        conn.commit()

# --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
# Writers follow the repo convention (swallow + logger.error, return bool); the

```

10. assistant (2026-06-22T15:05:27.297Z)

Confirmed: `query\_play\_segments` (line 564-612) has NO `ORDER BY` clause, while `query\_candidate\_segments` (line 524) explicitly adds `ORDER BY start\_time`. The finding's claims about these two functions are accurate.

Now let me verify the key claims:

1. The in-process path returns the segmenter's own ordered result directly (vs the cloud path round-trips through the DB).
2. Whether `intervals.csv` is actually start-ordered, and whether the reel ordering actually depends on this.

Let me check the in-process path's response shape and how `\_finalize\_reingest` works and how the compile/reel path consumes segments.

11. user (2026-06-22T15:05:28.374Z)

```

# Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
# if the run yields segments; otherwise drop the new partial rows and keep the old.
play_wm, cand_wm = db_instance.segment_watermarks(video_id)
# COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
# DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
# DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
# byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
# they're dropped on the cloud path in this slice ? a documented follow-up.)
remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
if remote is not None:
    segmentation_ran = True
    response = remote
    return response
segmenter = segmenter_cls(db_instance, global_config)
try:
    result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
except Exception:
    # A raising segmenter must not leave half-written rows: restore the pre-run
    # state (same destroy-after-confirm contract as the Failure branch), then let
    # the job worker record the failure.
    db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
    raise
segmentation_ran = True

```

```

if isinstance(result, Failure):
    logger.error(f"[API] Segmenter failed: {result.message}")
    db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
    response = {
        "video_id": video_id,
        "status": "failed",
        "message": f"Segmenter '{seg_name}' failed: {result.message}",
        "results": [r.model_dump() for r in val_results],
        "play_segments": [],
    }
    return response

segments = result.value if hasattr(result, "value") else result
notify("finalizing")
_finalize_reingest(video_id, segments, play_wm, cand_wm)

response = {
    "video_id": video_id,
    "status": "success",
    "message": f"Successfully indexed {len(segments)} play segments using '{seg_name}'
segmenter.",
    "results": [r.model_dump() for r in val_results],
    "play_segments": segments,
}
return response
finally:
    # Always emit the per-video observability report (next to the video, or to
    # report.output_dir). Never raises. Surface the paths in the response.
    paths = emit_indexer_report(
        db=db_instance, video_id=video_id, video_path=req.video_path, config=global_config,
        val_results=val_results, val_context=val_context,
        segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
        was_reingest=was_reingest, segmentation_ran=segmentation_ran,
    )
    if paths and isinstance(response, dict):
        response["reports"] = paths

@app.post("/api/process", status_code=202)
def process_video(req: ProcessRequest, request: Request):
    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

    Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
    client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
    runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
    """
    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing or
    # spoofed identity header fails here with 401, before any work) and tags the job's owner_id.
    try:
        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
    except edge_auth.EdgeAuthError as e:
        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root", None)
    try:
        validate_input_path(req.video_path, allowed_root=allowed_root)
        # M2: resolve the input once (path or storage URI) ? raises ValueError for an
        unsupported
        # scheme, which we map to a clean 400 (an unknown scheme must never 500).

```

12. assistant (2026-06-22T15:05:32.404Z)

Confirmed claim 1: the in-process path sets `play\_segments` to `result.value` (the segmenter's

own ordered list), while the cloud path rebuilds from `query\_play\_segments`. Now let me verify how the reel is actually compiled ? does the reel ordering use `play\_segments` from this response, or does it re-query the DB? And is `intervals.csv` actually start-ordered? Let me find `\_write\_intervals\_csv` and the compile path.

13. user (2026-06-22T15:05:34.248Z)

```
backend/annotations/automated_provider.py:89:         intervals = sorted((float(s), float(e)) for
s, e in intervals)
backend/annotations/base.py:57:         """Return the completed annotations as sorted `(start,
end)` intervals.
backend/api/uploads.py:75:         detail=f"Extension '{ext}' not allowed.
Allowed: {sorted(extensions)}")
backend/compute/base.py:38:         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` /
``report.json`` landed."""
backend/config/loader.py:146:         cfg_path, ", ".join(sorted(unknown)),
backend/eval/ablation.py:189:         def sorted_rows(self) -> List[AblationRow]:
backend/eval/ablation.py:191:             return sorted(self.rows, key=lambda r: r.delta_f1,
reverse=True)
backend/eval/ablation.py:202:             "rows": [r.as_dict() for r in self.sorted_rows()],
backend/eval/ablation.py:223:         dropped_label = ", ".join(sorted(cols)) or (gate_knob or "?")
backend/eval/ablation.py:317:         """Loop the inventory at the configured granularity ? an
``AblationReport`` (sorted table).
backend/eval/ablation.py:419:         rows = report.sorted_rows()
backend/eval/ablation_pdf.py:62:         rows: List[AblationRow] = report.sorted_rows()
backend/eval/annotations.py:51:         surface immediately rather than silently skewing scores.
Returns intervals sorted by
backend/eval/annotations.py:55:         intervals = load_gt_intervals(csv_path, strict=True)
backend/eval/audio/e1_probe.py:30:         ts = sorted(h.t for h in hits)
backend/eval/calibrate_wasb.py:41:         return load_gt_intervals(csv_path, strict=False)
backend/eval/eval_partial_labels.py:171:         rows.sort(key=lambda x: (x["f1"], x["mean_iou"]),
reverse=True)
backend/eval/eval_stats.py:49:         means.sort()
backend/eval/experiment.py:153:         blob = json.dumps(self.to_dict(), sort_keys=True,
default=str)
backend/eval/experiment.py:235:         for path in sorted(glob.glob(os.path.join(features_dir,
"*_golden_features.json"))):
backend/eval/fusion_golden.py:136:         specs = sorted(specs, key=lambda s:
(os.path.getsize(s["trajectory"]))
backend/eval/gemini_refine.py:47:         for s, e in sorted(intervals):
backend/eval/golden_fixtures.py:212:         rows.sort(key=lambda r: r[0])
backend/eval/golden_fixtures.py:478:         targets = [d.name for d in
sorted(fixtures_dir.iterdir())
backend/eval/golden_fixtures.py:519:         entries.sort(key=lambda e: str(e.get("slug")))
backend/eval/golden_fixtures.py:609:         out.append(f"(top-level): keys {sorted(recomputed)}
!= {sorted(committed)} (exact)")
backend/eval/golden_fixtures.py:625:         out.append(f"candidates[{i}].shuttle: keys
{sorted(rs)} != {sorted(cs)} (exact)")
backend/eval/golden_fixtures.py:644:         out.append(f"score: keys {sorted(rsc)} !=
{sorted(csc)} (exact)")
backend/eval/golden_fixtures.py:657:# Small JSON / path helpers (deterministic writers:
sort_keys, LF, trailing newline)
backend/eval/golden_fixtures.py:662:         text = json.dumps(obj, indent=2, sort_keys=True,
ensure_ascii=False)
backend/eval/golden_real_fixtures.py:140:         rows.sort(key=lambda r: r[0])
backend/eval/gt_loader.py:44: def load_gt_intervals(csv_path: str, *, strict: bool = True) ->
List[Interval]:
backend/eval/gt_loader.py:52:         Returns intervals sorted by start; overlapping GT intervals are
logged as a warning
backend/eval/gt_loader.py:92:         intervals.sort(key=lambda iv: iv[0])
backend/eval/harness.py:48:         """Pull (start_time, end_time) out of DB segment rows, sorted by
start."""
backend/eval/harness.py:50:         ivs.sort(key=lambda iv: iv[0])
backend/eval/harness.py:60:         rows = db.query_play_segments(video_id, {})
backend/eval/metrics.py:66:         candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
```

```

backend/eval/per_user_regression.py:157:         json.dump(baselines, f, indent=2,
sort_keys=True)
backend/eval/rally_seg_eval.py:412:         raise SystemExit(f"unknown preset '{name}'; choices:
{sorted(PRESETS)}")
backend/eval/rally_seg_eval.py:475:                                     "cap at its midpoint + flag/log it;
the continuously-active blob last resort)")
backend/eval/regression.py:39:         norm = sorted((round(float(s), 3), round(float(e), 3)) for s,
e in ground_truth)
backend/eval/regression.py:98:         json.dump(baselines, f, indent=2, sort_keys=True)
backend/eval/score_calibration.py:56:         ``xs`` are sorted training scores, ``ys`` the non-
decreasing fitted probabilities;
backend/eval/score_calibration.py:127:         """Fit isotonic (monotone) calibration via PAVA on
score-sorted labels."""
backend/eval/score_calibration.py:131:         order = sorted(range(n), key=lambda i: (scores[i], i))
backend/eval/segmentation_metrics.py:68:         order = sorted(range(len(scored_preds)),
backend/eval/splits.py:85:         f"Train/eval leakage detected on match ids:
{sorted(overlap)}. "
backend/eval/tolerance_metrics.py:46:                                     # sort key: combined error, then (i, j) for
a fully deterministic tiebreak
backend/eval/tolerance_metrics.py:48:         pairs.sort()

```

14. assistant (2026-06-22T15:05:39.391Z)

Interesting ? `backend/eval/harness.py:48-60` reads `query\_play\_segments` and explicitly re-sorts by start. That's evidence the codebase already knows `query\_play\_segments` is unordered. Let me find where the reel is actually compiled and whether ordering matters there. Let me find the compile endpoint and how it sources/orders segments.

15. user (2026-06-22T15:05:41.835Z)

```

backend/main.py:44:from backend.pipeline.stitcher.compiler import VideoCompiler
backend/main.py:45:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
backend/main.py:55:     StitchingConfig,
backend/main.py:281:     # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes
the reel watermark
backend/main.py:383:def _reels_dir() -> str:
backend/main.py:387:_REEL_EXTS = ("mp4", ".mov")
backend/main.py:390:@app.get("/api/reels")
backend/main.py:391:def list_reels():
backend/main.py:392:     """List compiled highlight reels present in the output dir, newest
first, with a download URL.
backend/main.py:394:     NB: reels are stored FLAT in output_dir under the bare name passed to
/api/compile ? the engine
backend/main.py:395:     does NOT track a video_id?reel association today (a per-video listing
would be a phantom), so
backend/main.py:396:     this is a global list. (Per-video reel tracking would need a DB record +
compile change ? a
backend/main.py:398:     base = _reels_dir()
backend/main.py:399:     reels = []
backend/main.py:403:         return {"reels": []}
backend/main.py:406:         if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
backend/main.py:411:             reels.append({
backend/main.py:415:                 "download_url": f"/api/reels/{quote(e.name)}",
backend/main.py:417:             reels.sort(key=lambda r: r["modified_at"], reverse=True)
backend/main.py:418:             return {"reels": reels}
backend/main.py:421:@app.get("/api/reels/{filename}")
backend/main.py:422:def download_reel(filename: str):
backend/main.py:423:     """Stream a compiled reel by its bare filename (no video_id ? reels are
flat in output_dir).
backend/main.py:429:     base = _reels_dir()
backend/main.py:435:     if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
backend/main.py:436:         raise HTTPException(status_code=404, detail=f"No compiled reel named
'{name}'.")
backend/main.py:475:     THIS DB, so the rest of the flow (query ? compile/local-stitch) is
unchanged.

```

```

backend/main.py:772:     """Stream a compiled highlight reel ? `filename` is the bare name given
to /api/compile
backend/main.py:806: def compile_highlights(req: CompileRequest):
backend/main.py:816:     raise HTTPException(status_code=400, detail="No matching play
segments found to stitch.")
backend/main.py:825:     # The configured shot-count floor (stitching.min_shot_count) applies on
the API
backend/main.py:826:     # path too ? parity with scripts/run_process_and_stitch.py. Segments
WITHOUT shot_count
backend/main.py:829:     stitching = getattr(global_config, "stitching", None) or
StitchingConfig()
backend/main.py:835:         if sc is not None and int(sc) < stitching.min_shot_count:
backend/main.py:841:         logger.info(f"[compile]
stitching.min_shot_count={stitching.min_shot_count}: dropped ")
backend/main.py:846:         f"stitching.min_shot_count={stitching.min_shot_count}.")
backend/main.py:851:     # Run compiler with the configured stitching paddings + optional
branding watermark, localized to
backend/main.py:852:     # the UI locale the SPA rendered in (req.locale): the reel's watermark
reads in the user's language
backend/main.py:855:     localized_watermark = localize_watermark(stitching.watermark,
req.locale)
backend/main.py:856:     compiler = VideoCompiler(output_dir,
begin_padding=stitching.begin_padding,

```

16. assistant (2026-06-22T15:05:45.465Z)

The reel compile path is a separate `/api/compile` endpoint. Let me read it to see how it sources segments ? this is the critical question of whether reel ordering actually depends on the cloud-path's `play\_segments` ordering, or whether compile re-queries.

17. user (2026-06-22T15:05:46.667Z)

```

def compile_highlights(req: CompileRequest):
    video = db_instance.get_video(req.video_id)
    if not video:
        raise HTTPException(status_code=404, detail="Indexed video record not found.")

    # Retrieve matching play segments from db
    all_segments = db_instance.query_play_segments(req.video_id, {})
    matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

    if not matched_segments:
        raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

    # Reject path traversal / absolute output names (os.path.join would otherwise let an
# absolute or '..'-laden output_filename write anywhere on disk).
    try:
        out_name = safe_output_filename(req.output_filename)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e

    # The configured shot-count floor (stitching.min_shot_count) applies on the API
    # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT shot_count
    # metadata are exempt: the floor filters known counts only, so explicitly
    # requested manual/legacy segments can't silently vanish under the default floor.
    stitching = getattr(global_config, "stitching", None) or StitchingConfig()
    kept = []
    for s in matched_segments:
        meta = s.get("metadata") or {}
        sc = meta.get("shot_count") if isinstance(meta, dict) else None
        try:
            if sc is not None and int(sc) < stitching.min_shot_count:
                continue

```

```

        except (TypeError, ValueError):
            pass # unparseable count -> treat as unknown, keep
        kept.append(s)
    if len(kept) < len(matched_segments):
        logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}: dropped "
                    f"{len(matched_segments) - len(kept)}/{len(matched_segments)} segments below
the floor.")
    if not kept:
        raise HTTPException(status_code=400,
                            detail=f"All {len(matched_segments)} matching segments fall below "
                                    f"stitching.min_shot_count={stitching.min_shot_count}.")

    # Extract start/end time pairs
    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

    # Run compiler with the configured stitching paddings + optional branding watermark,
    localized to
    # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the user's
    language
    # with a script-appropriate font. Unknown/absent locale keeps the configured default
    unchanged.
    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
    "output"
    localized_watermark = localize_watermark(stitching.watermark, req.locale)
    compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                             end_padding=stitching.end_padding, watermark=localized_watermark)
    try:
        # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://, gdrive://)
    for a
        # video ingested via /api/process from a URI, or a plain local path. Resolve it to a
    LOCAL file
        # the sticher can read: identity for a local path (byte-identical to before), fetch-to-
    scratch
        # for a bucket URI (the same seam the process path uses, _execute_processing
    main.py:159).
        # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a gs:// path.
        local_src = storage.acquire_local_input(video["filepath"], getattr(global_config,
    "storage", None))
        out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        # `filepath` is server-local (not reachable by the browser); also return the ready-to-
    use
        # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md $4.3).
        download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
        return {"status": "success", "filepath": out_path, "download_url": download_url}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Highlight stitching failed: {str(e)}")
    from e

```

--- CLI Debug Utility & Orchestration ---

```

def run_cli_diagnose_clip(video_path: str, timestamp: float):
    """CLI debugging utility to examine segment properties around a timestamp."""

```

18. assistant (2026-06-22T15:05:57.583Z)

This is the key finding. The actual `/api/compile` endpoint (which builds the reel) ALSO sources segments via `db\_instance.query\_play\_segments(req.video\_id, {})` ? the SAME unordered query ? and then builds `time\_pairs` from `kept` in whatever order the query returns. So the reel ordering at compile time depends on `query\_play\_segments`'s implicit ordering regardless of which path (in-process or cloud) produced the rows.

This is an important nuance for the finding's framing. Let me reconsider:

The finding claims the *cloud reel ordering* is fragile because the cloud path round-trips through `query\_play\_segments`. But the reel is actually compiled by `/api/compile`, which ALWAYS

goes through `query\_play\_segments` ? for BOTH the in-process and cloud paths. So:

- The in-process path's response `play\_segments` (from `result.value`) is only used to *populate the UI list / response*, not directly to build the reel.
- The reel itself is built by `/api/compile` from `query\_play\_segments` in BOTH cases.

So the implicit-ordering dependency in `/api/compile` is pre-existing (not introduced by this PR) and applies to both paths. The new cloud-path code at line 522 only affects the `play\_segments` returned in the `/api/process` response (the UI list ordering shown after processing).

Let me check whether the SPA / UI relies on the response `play\_segments` order, and whether the segment selection order is preserved in `req.segment\_ids`. Let me look at the CompileRequest and whether `time\_pairs` order matters at all for the reel.

19. user (2026-06-22T15:05:58.242Z)

```
261:class ProcessRequest(BaseModel):
277:class CompileRequest(BaseModel):
279:    segment_ids: List[int]
813:    matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
---compiler---
import os
import subprocess
import tempfile
import logging
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple, Union

from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe resolver

logger = logging.getLogger(__name__)
```

Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the over renders out-of-the-box without a system fontconfig setup (a common gap on Windows `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives a backend/pipeline/stitcher/, so parents[2] == backend/.

```
_BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
```

```
class VideoCompiler:
    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
                 watermark: Optional[Any] = None):
        self.output_dir = output_dir
        self.end_padding = end_padding
        self.begin_padding = begin_padding
        # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
        # normalized to a dict (or None) so this module stays config-package-agnostic.
        self.watermark = self._normalize_watermark(watermark)
        os.makedirs(output_dir, exist_ok=True)

    @staticmethod
    def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
        if watermark is None:
            return None
        if hasattr(watermark, "model_dump"):
            return watermark.model_dump()
        if isinstance(watermark, dict):
            return dict(watermark)
        return None

    @staticmethod
```

```

def _ff_quote(path: str) -> str:
    """Make a filesystem path safe to embed inside single quotes in an ffmpeg
    filtergraph operand (drawtext fontfile/textfile, movie= source): forward
    slashes (Windows-friendly), escaped single quotes, AND escaped colons.

    The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
    option parser treats ':' as the option separator, so a Windows drive-letter
    path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
    the encode fails with "Invalid argument". This silently disabled the
    on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
    return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")

@staticmethod
def _parse_time(val: Union[int, float, str]) -> float:
    """Normalizes a timestamp value to float seconds.
    Handles: float/int (pass-through), plain numeric strings ("12.4"),
    and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
    This mirrors the parse_time logic used in gemini.py to ensure
    the stitcher can always consume whatever format the indexer produced."""
    if isinstance(val, (int, float)):
        return float(val)

```

20. assistant (2026-06-22T15:06:02.465Z)

Now let me check whether `compile\_highlights` builds `time\_pairs` in a sorted order or relies on the query order. Looking back at line 813: `matched\_segments = [s for s in all\_segments if s["id"] in req.segment\_ids]` ? this preserves `all\_segments` (query) order, NOT `req.segment\_ids` order. And then `time\_pairs` is built from `kept` in that same order. Let me check whether the compiler internally sorts the time_pairs.

21. user (2026-06-22T15:06:04.074Z)

```

83:     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float, float]]:
84:         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
85:         for raw_s, raw_e in segments:
86:             parsed.sort()
87:             # without fontconfig) ? fontconfig family (last-resort, environment-dependent).
88:             segments: List[Tuple[float, float]],
89:             None and the (idx, start, end, reason) record to record in skipped_segments."""
90:             segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
91:             def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
92: output_filename: str) -> str:
93:                 Extracts selected segments from a raw video and stitches them together
94:                 if not segments:
95:                     raise ValueError("No highlight segments provided to compile.")
96:                 # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
97:                 original_count = len(segments)
98:                 segments = self._merge_overlapping(segments)
99:                 if len(segments) < original_count:
100:                     original_count - len(segments), original_count, len(segments))
101:                 if not segments:
102:                     raise ValueError("No valid (start < end) highlight segments after merge.")
103:                 # Probe video duration so we can detect out-of-range segments
104:                 skipped_segments = []
105:                 logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
106:                 for idx, (raw_start, raw_end) in enumerate(segments):
107:                     idx, raw_start, raw_end, segments, video_duration,
108:                     skipped_segments.append(skip_record)
109:                 # Report summary of skipped segments
110:                 if skipped_segments:
111:                     logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
112:                     for seg_idx, seg_start, seg_end, reason in skipped_segments:
113:                         f"All {len(segments)} segments were skipped. "

```

22. assistant (2026-06-22T15:06:11.106Z)

This is the decisive find. The `VideoCompiler.compile_highlights` calls `self._merge_overlapping(segments)` at line 316, which at line 84/100 **normalizes, sorts** (`parsed.sort()`), and merges** the segments. So the compiler internally re-sorts the time pairs by start before stitching.

This means: **the reel ordering does NOT depend on the order of `play_segments` / `query_play_segments` at all.** The compiler sorts the `(start, end)` pairs itself before producing the reel. The finding's central claim ? "the cloud reel ordering therefore depends on the implicit rowid==insertion invariant holding" and "a non-start-ordered intervals.csv would silently scramble the cloud reel order" ? is **false**. Even if `query_play_segments` returned rows in arbitrary order, the compiler's `_merge_overlapping` would re-sort them, so the reel comes out start-ordered regardless.

Let me confirm `_merge_overlapping` sorts and that nothing in the compile path bypasses it.

23. user (2026-06-22T15:06:12.382Z)

```
def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float, float]]:
    """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
```

```

    A highlight reel must never replay the same footage. The same rally can land in
    the segment list more than once ? e.g. two detector runs accumulated for one
    video (add_video is an UPSERT, so re-running without a prune leaves both sets), or
    chunk-overlap duplicates near boundaries. Stitching those verbatim plays the rally
    twice. Merging any range that overlaps or abuts the previous one collapses true
    duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
    Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
    """
```

```
    parsed = []
    for raw_s, raw_e in segments:
        s = VideoCompiler._parse_time(raw_s)
        e = VideoCompiler._parse_time(raw_e)
        if e > s:
            parsed.append((s, e))
    parsed.sort()
    merged: List[Tuple[float, float]] = []
    for s, e in parsed:
        if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))
    return merged
```

```
def _get_video_duration(self, video_path: str) -> float:
    """Probes the video file to get its total duration in seconds.
```

24. assistant (2026-06-22T15:06:18.075Z)

Confirmed. `_merge_overlapping` does `parsed.sort()` (line 100), which sorts `(start, end)` tuples ascending by start. The compiler is the single chokepoint for reel ordering and it always sorts. So the reel is **always** start-ordered, independent of `query_play_segments`'s row order or `intervals.csv` order.

Let me do one final check on the other consumer surface the finding mentions ? the `play_segments` returned in the `/api/process` response itself (the UI list). Even though the reel is safe, does the UI list-order matter and is it now inconsistent between paths? Let me check whether the in-process path's `result.value` is even guaranteed start-ordered (to assess whether there's a real cross-path *consistency* concern, separate from the reel).

25. user (2026-06-22T15:06:20.601Z)

<persisted-output>

Output too large (312.5KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\tool-results\bb6iwh1rz.txt

Preview (first 2KB):

```
backend/ui/assets/index-GSXqL6iY.js:25: */var Lh;function Ub(){return
Lh||(Lh=1,(function(i){function l(N,U){var I=N.length;N.push(U);e:for(;0<I;){var
me=I-1>>>1,ke=N[me];if(0<c(ke,U))N[me]=U,N[I]=ke,I=me;else break e}}function r(N){return
N.length===0?null:N[0]}function u(N){if(N.length===0)return null;var
U=N[0],I=N.pop();if(I!==U){N[0]=I;e:for(var me=0,ke=N.length,S=ke>>>1;me<S;){var w=2*(me+1)-
1,G=N[w],Z=w+1,re=N[Z];if(0>c(G,I))Z<ke&&0>c(re,G)?(N[me]=re,N[Z]=I,me=Z):(N[me]=G,N[w]=I,me=w);
else if(Z<ke&&0>c(re,I))N[me]=re,N[Z]=I,me=Z;else break e}}return U}function c(N,U){var
I=N.sortIndex-U.sortIndex;return I!==0?I:N.id-U.id}if(i.unstable_now=void 0,typeof
performance=="object"&&typeof performance.now=="function"){var
f=performance;i.unstable_now=function(){return f.now()}}else{var
g=Date,h=g.now();i.unstable_now=function(){return g.now()-h}}var
p=[],b=[],v=1,y=null,A=3,L=!1,j=!1,B=!1,V=!1,le=typeof
setTimeout=="function"?setTimeout:null,Y=typeof
clearTimeout=="function"?clearTimeout:null,Q=typeof setImmediate<"u"?setImmediate:null;function
$(N){for(var U=r(b);U!==null;){if(U.callback===null)u(b);else
if(U.startTime<=N)u(b),U.sortIndex=U.expirationTime,l(p,U);else break;U=r(b)}}function
ne(N){if(B=!1,$(N),!j)if(r(p)!==null)j=!0,ge||(ge=!0,P());else{var
U=r(b);U!==null&&fe(ne,U.startTime-N)}}var ge=!1,X=-1,te=5,z=-1;function q(){return
V?!0:!(i.unstable_now()-z<te)}function ie(){if(V=!1,ge){var N=i.unstable_now();z=N;var
U=!0;try{e:{j=!1,B&&(B=!1,Y(X),X=-1),L=!0;var
I=A;try{t:{for($(N),y=r(p);y!==null&&!(y.expirationTime>N&&q());){var me=y.callback;if(typeof
me=="function"){y.callback=null,A=y.priorityLevel;var
ke=me(y.expirationTime<=N);if(N=i.unstable_now(),typeof
ke=="function"){y.callback=ke,$(N),U=!0;break t}y===r(p)&&u(p,$(N))else
u(p);y=r(p)}if(y!==null)U=!0;else{var S=r(b);S!==null&&fe(ne,S.startTime-N),U=!1}}break
e}}finally{y=null,A=I,L=!1}U=void 0}}finally{U?P():ge=!1}}}}var P;if(typeof
Q=="function")P=function(){Q(ie)};else if(typeof MessageChannel<"u")
...
</persisted-output>
```

26. assistant (2026-06-22T15:06:23.803Z)

That hit the minified bundle. Let me check the test file directly for any ordering assertions, which tells me whether the PR author considered order a contract.

27. user (2026-06-22T15:06:25.638Z)

```
4:intervals back into the API DB, so /api/query + /api/compile (local stitch) are unchanged ?
all
5:GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv +
the
39:      ""Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv + the
done-marker
58:      with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
60:          w.writerow(["start", "end", "shot_count", "ending_reason"])
113:      assert len(out["play_segments"]) == 2
114:      rows = db.query_play_segments(out["video_id"], {})
149:      assert out["play_segments"] == []
150:      assert db.query_play_segments(out["video_id"], {}) == []
182:      return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
187:      assert len(out["play_segments"]) == 1
210:      segs = q.json()["play_segments"]
```

28. assistant (2026-06-22T15:06:33.335Z)

The test asserts on `len(play\_segments)` and contents, not on order. No ordering contract is asserted. Let me do a final confirmation that I've understood the complete picture, and verify

the `harness.py` re-sort (which demonstrates the codebase already treats `query\_play\_segments` as unordered and sorts when order matters ? meaning the established convention is "sort at the consumer that needs order").

29. user (2026-06-22T15:06:34.222Z)

```
def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
    """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
    ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
    ivs.sort(key=lambda iv: iv[0])
    return ivs

def get_predictions(db: Database, video_id: str, stage: str,
                    detector: Optional[str] = None) -> List[Interval]:
    """Fetch predicted rally intervals for one stage from the DB."""
    if stage == "candidates":
        rows = db.query_candidate_segments(video_id, detector=detector)
    elif stage == "final":
        rows = db.query_play_segments(video_id, {})
    else:
        raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
```

30. assistant (2026-06-22T15:06:45.987Z)

I now have the complete picture. Let me record my verdict.

Verdict

****False alarm (nit at best).**** The finding's factual sub-claims are accurate, but its central impact claim ? that the **cloud reel ordering** is fragile and would "silently scramble" if `intervals.csv` weren't start-ordered ? does not hold up.

****What's true:****

- `query\_play\_segments` (database.py:564-612) has no `ORDER BY`; `query\_candidate\_segments` (database.py:524) does add `ORDER BY start\_time`. Accurate.
- The cloud path rebuilds `play\_segments` via `query\_play\_segments` (main.py:522) while the in-process path returns the segmenter's own `result.value` (main.py:633). Accurate.

****Why the impact claim fails (the load-bearing error):****

- The reel is built by `/api/compile` (`compile\_highlights`, main.py:806), which sources segments via `db\_instance.query\_play\_segments(req.video\_id, {})` ? the SAME unordered query ? for BOTH the in-process and cloud paths. The new line 522 only populates `play\_segments` in the `/api/process` **response** (the post-process UI list), not the reel.
- More decisively, `VideoCompiler.compile\_highlights` (compiler.py:316) passes the `(start,end)` pairs through `\_merge\_overlapping`, which does `parsed.sort()` (compiler.py:100) before stitching. The compiler is the single chokepoint for reel ordering and it ****always re-sorts by start****. So the reel is start-ordered regardless of `query\_play\_segments`' row order or `intervals.csv` order. A scrambled `intervals.csv` would NOT scramble the reel.

****Supporting evidence:**** The codebase's established convention is to sort at the consumer that needs order, not in `query\_play\_segments` ? e.g. `backend/eval/harness.py:48` re-sorts `query\_play\_segments` rows by start. The new test (`tests/test\_api\_cloud\_dispatch.py`) asserts on segment count/content, not order ? no ordering contract exists.

The only residual is a cosmetic cross-path inconsistency in the `/api/process` response `play\_segments` list order (segmenter order vs DB scan order), with no functional consequence to the reel and no asserted contract. That's a nit, not a defect, and the PR did not introduce the unordered `query\_play\_segments`.

31. user (2026-06-22T15:06:52.625Z)

Structured output provided successfully

32. assistant (2026-06-22T15:07:00.014Z)

I've verified the finding and returned my structured verdict.

****Summary: False alarm (nit).****

The finding's factual sub-claims check out, but its core impact claim collapses under inspection:

- `C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\stitcher\compiler.py:316` routes segments through `\_merge\_overlapping`, which calls `parsed.sort()` at `compiler.py:100` before stitching. The compiler is the sole chokepoint for reel ordering and ****always**** re-sorts by start ? so a scrambled `intervals.csv` or unordered `query\_play\_segments` cannot scramble the cloud reel.
- The reel is built by `/api/compile` (`backend/main.py:806`), which re-queries via the same unordered `query\_play\_segments` for both the in-process and cloud paths ? line 522's rebuild only feeds the response/UI list, not the reel.
- The unordered `query\_play\_segments` is pre-existing, and the codebase convention is to sort at the consumer (e.g. `backend\eval\harness.py:48`). The new test asserts on count/content, not order.

The only residual is a cosmetic cross-path inconsistency in the `/api/process` response list order, with no functional consequence.